Assignment 7B

Aim - Write a menu driven program to implement queue using linked list

Theory

Queue is a linear data structure which is implemented using array or linked list.

Enqueue (insert) - It adds an element (new node) to the queue if the queue is empty, both front and rear point to the new node or set rear's next to the new node and update rear to the new node.

Dequeue (Delete) - It deletes an element and increments the value of front. It basically frees the node of the linked list containing the element.

Display - It prints all elements of the queue traversing from front till NULL and print each node.

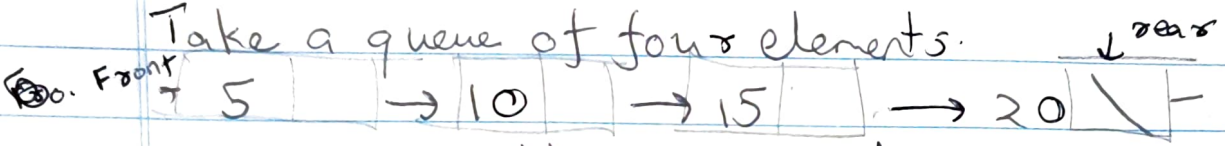
Time complexity - The time complexity is O(n) as all elements need to be taversed

Conclusion

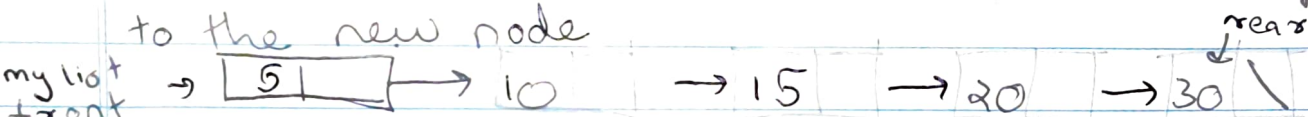
The concept of linked lists is applied to evaluate problems in a variety of applications.

Working Example

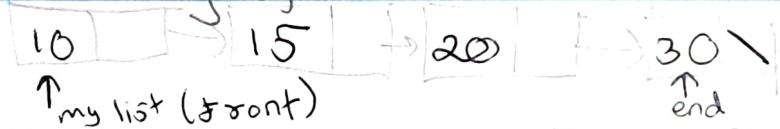
Take a queue of four elements.



to enqueue, add a new node and points it to the end element and initialize front to the new node and rear to the end element node. The last element points to the new node



To dequeue, increment front by one and free the memory of that node



```
#include <stdio.h>
#include <stdlib.h>

struct ll_node
{
   int data;
   struct ll_node *next;
};

void enqueue(struct ll_node **front, struct ll_node **rear, int val);
void dequeue(struct ll_node **front, struct ll_node **rear);
void display(struct ll_node *front);

void main()
{
   int val, choice;
   struct ll_node *front = NULL;
   struct ll_node *rear = NULL;

   do
   {
      printf("--------------MENU----------------\n");
      printf("1.Enqueue\n2.Dequeue\n3.Display\n4.Exit\nEnter your choice: ");
      scanf("%d", &choice);

      switch (choice)
      {
      case 1:
```

```
        printf("Enter the value to be enqueued: ");
        scanf("%d", &val);
        enqueue(&front, &rear, val);
        break;

      case 2:
        dequeue(&front, &rear);
        break;

      case 3:
        display(front);
        break;

      case 4:
        printf("........exiting program\n");
        break;

      default:
        printf("Invalid choice\n");
      }

   } while (choice != 4);
}

void enqueue(struct ll_node **front, struct ll_node **rear, int val)
{
   struct ll_node *pnew = (struct ll_node *)malloc(sizeof(struct ll_node));
```

```
    if (pnew == NULL)
    {
        printf("Memory allocation failed\n");
        return;
    }

    pnew->data = val;
    pnew->next = NULL;

    if (*front == NULL)
    {
        *front = pnew;
        *rear = pnew;
        return;
    }

    (*rear)->next = pnew;
    *rear = pnew;
}

void dequeue(struct ll_node **front, struct ll_node **rear)
{
    struct ll_node *node2delete;

    if (*front == NULL)
    {
        printf("Queue underflow\n");
        return;
```

```
    }

    node2delete = *front;
    *front = (*front)->next;

    if (*front == NULL)
    {
        *rear = NULL;
    }

    free(node2delete);
}

void display(struct ll_node *front)
{
    struct ll_node *help_pointer = front;

    if (help_pointer == NULL)
    {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue contains:\n");

    while (help_pointer != NULL)
    {
        printf("%d\t", help_pointer->data);
```

```
        help_pointer = help_pointer->next;
    }
    printf("\n");
}
```

```
--------------MENU----------------
1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter your choice: 1
Enter the value to be enqueued: 10
--------------MENU----------------
1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter your choice: 1
Enter the value to be enqueued: 20
--------------MENU----------------
1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter your choice: 1
Enter the value to be enqueued: 30
--------------MENU----------------
1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter your choice: 3
Queue contains:
10      20      30
```

```
--------------MENU----------------
1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter your choice: 2
--------------MENU----------------
1.Enqueue
2.Dequeue
3.Display
4.Exit
Enter your choice: 3
Queue contains:
20      30
```